# Práctica evaluable: Organizador automático de carpetas

**Resultado de aprendizaje evaluado:**

- **RA3.** Escribe código utilizando programación modular. Conoce la diferencia entre el paso de parámetros por valor y por referencia.

**Lenguaje de referencia:** Python 3

**Librería externa requerida:** `rich`

**Otras librerías utilizadas:** `os` y `shutil` — módulos de la biblioteca estándar de Python; no requieren instalación adicional.

## 1. Enunciado

Crea un programa en Python que **automatice la organización de los archivos de una carpeta**.

El programa pedirá al usuario una **carpeta de origen** y una **carpeta de destino**, y moverá a la carpeta de destino **todos los archivos de la carpeta de origen, incluidos los que estén dentro de subcarpetas (a cualquier nivel de profundidad)**, clasificándolos en subcarpetas de la carpeta de destino según su extensión (por ejemplo, todas las imágenes JPG en una subcarpeta `jpg`, todos los documentos PDF en `pdf`, etc.). Al terminar, generará un pequeño informe con el número de archivos movidos, el espacio total que ocupan (mostrado en una unidad legible: bytes, KB, MB o GB según corresponda), cuántos archivos quedan en la carpeta de origen y cuántos hay en total en la carpeta de destino.

Toda la interacción con el usuario (cabecera del programa, petición de datos, avisos de error, seguimiento de la clasificación e informe final) debe presentarse mediante una **consola enriquecida** creada con la librería `rich`: texto en colores, paneles y tablas.

El programa debe resolverse mediante **funciones definidas por el programador**, cada una con una responsabilidad concreta, aplicando lo estudiado en esta unidad sobre paso de parámetros y sobre recursividad.

Las funciones que **calculan o actúan sobre los archivos** no deben mostrar nada por pantalla: solo devuelven resultados o modifican sus parámetros; la presentación con `rich` se encarga a otras funciones y al programa principal.

## 2. Requisitos funcionales y técnicos

El programa debe cumplir, como mínimo, los siguientes requisitos:

1. **Consola y cabecera.** Al inicio del programa se debe crear **un único objeto `Console`** (`from rich.console import Console`), que se utilizará en lugar de `print()` para toda la salida por pantalla. Mediante una función `mostrar_cabecera(consola)` se mostrará el título del programa dentro de un `Panel`.
2. **Carpetas de origen y de destino.** El programa debe pedir por teclado, utilizando `Prompt.ask()` (`from rich.prompt import Prompt`) con un texto de petición en color, **la ruta de la carpeta de origen y la ruta de la carpeta de destino.** Antes de continuar deben realizarse tres comprobaciones y, si alguna falla, el programa debe avisar con un mensaje en **rojo** (por ejemplo, dentro de un `Panel` con `border_style="red"`) y finalizar sin dar error:
   - la carpeta de origen debe existir y ser una carpeta (`os.path.isdir()`);
   - si la ruta de destino ya existe, debe ser una carpeta (`os.path.isdir()`); si no existe, se creará automáticamente al clasificar el primer archivo (punto 7);
   - la carpeta de origen y la de destino no pueden ser la misma ni estar una dentro de la otra (se comprobará con la función del punto 5), para evitar volver a procesar archivos ya clasificados.
3. **Función `obtener_extension(nombre_archivo)`.** Debe devolver, en minúsculas y sin el punto, la extensión de un nombre de archivo (por ejemplo, `"Informe.PDF"` → `"pdf"`), apoyándose en `os.path.splitext()`. Si el archivo no tiene extensión, debe devolver el texto `"sin_extension"`.
4. **Función `tamano_legible(tamano_bytes)`.** Debe resolver un problema de cálculo elemental: convertir un tamaño en bytes a la unidad más adecuada (bytes, KB, MB o GB), dividiendo

sucesivamente entre 1024 mientras el valor lo permita, y devolver el resultado como una cadena de texto ya formateada (por ejemplo, `"2.35 MB"`). Debe probarse con varios valores de prueba que caigan en cada una de las cuatro unidades. El cálculo debe programarse a mano: **no se puede usar `rich.filesize`** ni ninguna otra utilidad que haga la conversión por sí sola.

5. **Función `una_dentro_de_otra(ruta_a, ruta_b)`.** Debe devolver `True` si las dos rutas son la misma carpeta o si una de ellas está dentro de la otra, y `False` en caso contrario. Se comparan las rutas absolutas (`os.path.abspath()`) a las que se les añade `os.sep` al final, de modo que, por ejemplo, `fotos` y `fotos2` no se consideren una dentro de otra, y se usa `.startswith()` en ambos sentidos.
6. **Función `nombre_disponible(ruta_destino, nombre_archivo)`.** Al reunir en una misma carpeta archivos procedentes de carpetas distintas, es posible que dos de ellos tengan el mismo nombre (por ejemplo, un `notas.txt` en cada carpeta). Esta función debe devolver un nombre que **no exista todavía** en `ruta_destino`: si `nombre_archivo` ya existe, se probará con `nombre_1.ext`, `nombre_2.ext`, etc. (separando nombre y extensión con `os.path.splitext()` y comprobando con `os.path.exists()` dentro de un bucle `while`), hasta encontrar uno libre. Así ningún archivo se sobrescribe ni se pierde.
7. **Función `clasificar_archivo(ruta_origen, nombre_archivo, ruta_destino, historial)`.** Debe mover un archivo a la subcarpeta de la carpeta de destino que le corresponda según su extensión (creando esa subcarpeta con `os.makedirs(..., exist_ok=True)` si todavía no existe, obteniendo un nombre libre con `nombre_disponible()` y moviendo el archivo con `shutil.move()`), y debe además añadir a `historial`, mediante `.append()`, un texto que describa el movimiento (por ejemplo, `"carpeta_prueba/trabajos/notas.txt → txt/notas_1.txt"`). El parámetro `historial` debe ser una lista **ya creada antes de llamar a la función**, de forma que se demuestre que, al tratarse de un tipo mutable, la función modifica directamente la lista original (paso "por referencia"), sin necesidad de devolverla con `return`. La función devolverá con `return` el tamaño en bytes del archivo movido (obtenido con `os.path.getsize()` antes de moverlo).
8. **Función recursiva `clasificar_carpeta_recursivo(ruta_origen, ruta_destino, historial)`.** Debe recorrer la carpeta de origen **y todas sus subcarpetas, a cualquier profundidad**, clasificando todos los archivos que encuentre. Recorrerá con un bucle `for` el contenido de la carpeta (`os.listdir()`): cuando un elemento sea una subcarpeta (`os.path.isdir()`), la función **se llamará a sí misma** con esa subcarpeta; cuando sea un archivo, llamará a `clasificar_archivo()`. **Se pasará la misma lista `historial` en todas las llamadas recursivas**, de modo que al final contenga los movimientos de todos los niveles. La función devolverá con `return` el total de bytes movidos (la suma de lo que devuelvan `clasificar_archivo()` y las llamadas recursivas). Identifica claramente el caso base y el caso recursivo (mediante comentarios). **No se puede usar `os.walk()`**, precisamente para tener que resolver el recorrido en profundidad mediante recursividad propia. Las carpetas de origen no se borran, aunque queden vacías.
9. **Función recursiva `contar_archivos_recursivo(ruta)`.** Debe recorrer una carpeta y todas sus subcarpetas, a cualquier profundidad, devolviendo el número total de archivos que contiene (de nuevo sin `os.walk()`). Se usará, una vez terminada la clasificación, para saber cuántos archivos quedan en la carpeta de origen (debería ser 0) y cuántos hay en la carpeta de destino (incluidos los que ya hubiera antes de la ejecución, si la carpeta existía).
10. **Programa principal e informe final con tablas.** El programa principal pedirá las carpetas (puntos 1 y 2), creará la lista `historial` y llamará a `clasificar_carpeta_recursivo()`; opcionalmente, puede envolver esta llamada en `consola.status()` para mostrar un indicador de progreso. Al terminar, el informe se mostrará mediante una función `mostrar_informe(consola, movidos, tamano_total, en_origen, en_destino, historial)`, que reciba la consola y los datos por parámetro (el número de archivos movidos puede obtenerse con `len(historial)`) y construya:
    - una **tabla** (`rich.table.Table`) con cuatro filas: archivos movidos, tamaño total movido (usando `tamano_legible()`), archivos que quedan en el origen y archivos en el destino (con subcarpetas). Se recomienda resaltar en verde el valor `0` de los archivos que quedan en el origen y en rojo cualquier valor mayor que cero;

- una **segunda tabla** con los movimientos guardados en `historial` (una fila por archivo, numeradas), recorriendo `historial` con un bucle `for`.

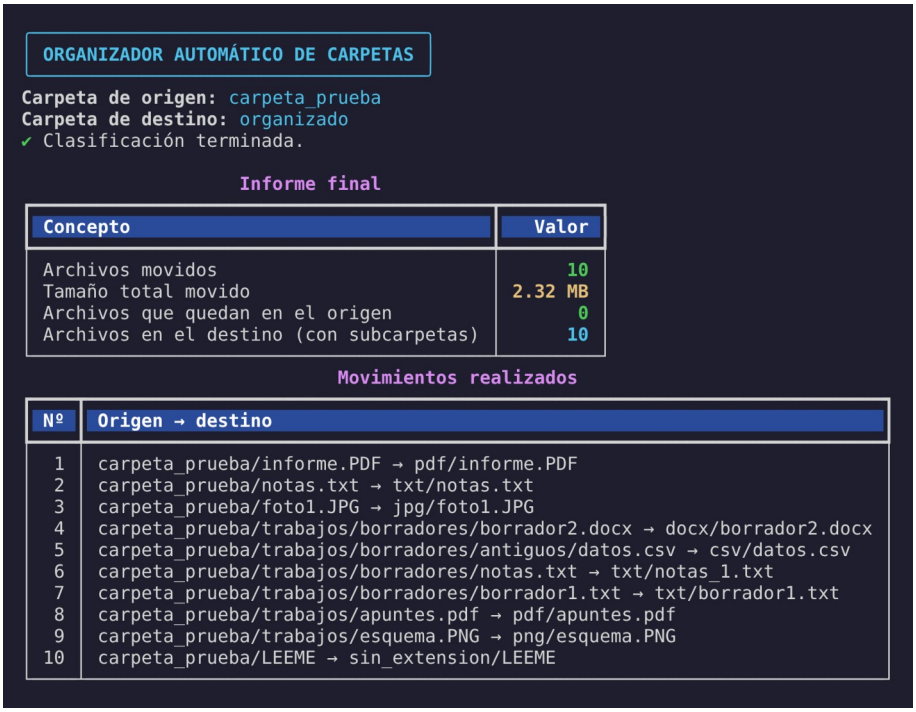
11. **Separación entre cálculo y presentación.** Solo las funciones de presentación (`mostrar_cabecera()`, `mostrar_informe()`) y el programa principal pueden utilizar `rich`. Las funciones de los puntos 3 a 9 deben limitarse a calcular y devolver resultados, a actuar sobre los archivos o a modificar sus parámetros, **sin imprimir nada** por pantalla.
12. **Documentación del código.** El archivo debe incluir un docstring de módulo que describa su propósito, un docstring en cada función definida (qué hace, qué parámetros recibe y qué devuelve) y comentarios que expliquen los bloques principales del programa.

## Ejemplo orientativo

Supongamos que la carpeta `carpeta_prueba` tiene, antes de ejecutar el programa, la siguiente estructura (con subcarpetas anidadas que contienen archivos, y con un `notas.txt` repetido en dos niveles distintos):

```
carpeta_prueba/
├── foto1.JPG
├── informe.PDF
├── LEEME
├── notas.txt
└── trabajos/
    ├── apuntes.pdf
    ├── esquema.PNG
    └── borradores/
        ├── borrador1.txt
        ├── borrador2.docx
        ├── notas.txt
        └── antiguos/
            └── datos.csv
```

Al ejecutar el programa con `carpeta_prueba` como origen y `organizado` como destino (carpeta que todavía no existe), la consola mostraría algo parecido a esto (el orden de los movimientos puede variar, porque `os.listdir()` no devuelve los nombres ordenados, y por tanto también puede variar cuál de los dos `notas.txt` recibe el sufijo `_1`):



Tras la ejecución, **todos los archivos están en la carpeta de destino, clasificados por extensión, y el `notas.txt` repetido se ha conservado con el nombre `notas_1.txt`**. En la carpeta de origen solo quedan las carpetas, ya vacías:

```
organizado/
├── csv/
│   └── datos.csv
├── docx/
│   └── borrador2.docx
├── jpg/
│   └── foto1.JPG
├── pdf/
│   ├── apuntes.pdf
│   └── informe.PDF
├── png/
│   └── esquema.PNG
├── sin_extension/
│   └── LEEME
└── txt/
    ├── borrador1.txt
    ├── notas.txt
    └── notas_1.txt

carpeta_prueba/
└── trabajos/
    └── borradores/
        └── antiguos/
```

Si alguna de las comprobaciones del punto 2 falla, el programa muestra el aviso en rojo y termina; por ejemplo, con el destino dentro del origen:

```
╭──────────────────────────────────────────────────────────────────────────────╮
│ Origen y destino no pueden ser la misma carpeta ni estar una dentro de la    │
│ otra.                                                                        │
╰──────────────────────────────────────────────────────────────────────────────╯
```

El formato exacto es libre, siempre que se utilicen paneles, colores y tablas de `rich`.

### Restricciones importantes

- **No se puede usar `os.walk()` ni ninguna otra función que resuelva el recorrido recursivo de carpetas por sí sola**; el objetivo de la práctica es que la recursividad la programe el propio alumnado.
- La librería `rich` **no puede sustituir** los cálculos que se evalúan: no se permite `rich.filesize` para `tamano_legible()`, ni ninguna utilidad de `rich` (como `rich.tree`) para resolver el recorrido de carpetas de los puntos 8 y 9.
- El objeto `Console` se creará una sola vez y se **pasará como parámetro** a las funciones de presentación, sin declararlo como variable global.

## 3. Entrega

1. Crear un proyecto en el IDE de trabajo (por ejemplo, Visual Studio Code), con su propio entorno virtual, e instalar en él la librería `rich` con `pip install rich`.
2. Crear el fichero `requirements.txt` que contenga las dependencias de vuestro entorno virtual.
3. Crear una **carpeta de pruebas** independiente (por ejemplo, `carpeta_prueba/`) con varios archivos de distintos tipos (imágenes, documentos, texto, archivos sin extensión...) y, para poder comprobar la recursividad, con **al menos dos niveles de subcarpetas anidadas que ya contengan archivos**

antes de ejecutar el programa, y con **al menos dos archivos con el mismo nombre en carpetas distintas** (para comprobar `nombre_disponible()`).

4. Probar el programa con varios casos de uso distintos, incluyendo al menos cuatro casos límite: una carpeta de origen vacía, una ruta de origen que no exista o no sea una carpeta, una carpeta de destino situada dentro de la de origen, y una carpeta origen y destino válidas (similar al ejemplo orientativo) que permitan probar correctamente el programa.
5. Realizar el desarrollo utilizando vuestro propio repositorio en GitHub, donde se refleje el historial de cambios desde el inicio del desarrollo y sucesivos commits según habeis ido avanzando en el desarrollo hasta llegar a vuestra solución final.
6. La entrega se realizará a través de la tarea del Aula Virtual, donde debeis:
   • Indicar la url de vuestro repositorio publico en GitHub que deberá contener tanto los ficheros .py como el `requirements.txt` que permita crear el entorno virtual en otro ordenador,
   • Adjuntar un video de, como mínimo, cuatro ejecuciones de prueba distintas que se correspondan con los casos de uso del punto (4) que habeis definido para probar vuestra solución. En el video se deberá poder observar tanto la entrada y salida de consola como el estado inicial y final de las carpetas en cada ejecución.

### 3.1. Condiciones relativas a la entrega

Con el fin de asegurar que el programa entregado es obra propia del alumno o alumna, se aplicarán las siguientes condiciones:

1. **Trabajo individual.** La práctica se realiza de forma individual. Queda prohibido compartir el código, total o parcialmente, con otros compañeros o compañeras, así como entregar código copiado de internet o de cualquier otra fuente sin citar. La consulta de la documentación oficial de `rich` y el uso de sus ejemplos básicos están permitidos, siempre citando la fuente.
2. **Desarrollo supervisado en el aula.** El desarrollo y pruebas se realizará en el aula, en horario de clase y bajo supervisión del profesorado, de forma que se pueda observar el proceso de trabajo real del alumnado.
3. **Defensa oral breve.** Cada alumno o alumna deberá defender su programa ante el profesor: explicar brevemente el funcionamiento de dos o tres funciones elegidas en el momento y realizar en directo una pequeña modificación guiada (por ejemplo, añadir una nueva categoría de extensión, cambiar la forma de renombrar los archivos repetidos o cambiar el formato del informe final), para comprobar que comprende lo que ha escrito. Una defensa que no permita verificar la comprensión del código puede suponer la no superación de la práctica, con independencia de la calidad de la solución entregada.
4. **Historial de desarrollo.** Se deberá utilizar un sistema de control de versiones (GitHub) desde el inicio del desarrollo, de forma que el historial de commits permita comprobar la evolución incremental del programa.
5. **Comprobación de similitud entre entregas.** Se compararán las entregas del grupo-clase (mediante una herramienta de detección de similitud de código o de forma manual). Un grado de coincidencia que no pueda justificarse (estructuras y nombres de variable idénticos, mismos comentarios, mismos errores...) invalidará la práctica para todas las personas implicadas, sin perjuicio de otras medidas disciplinarias que correspondan según la normativa del centro.
6. **Uso de asistentes de IA generativa.** No está permitido el uso de herramientas de inteligencia artificial generativa para escribir el código de la práctica, salvo que el profesorado indique expresamente lo contrario. Ante cualquier duda razonable sobre la autoría del código, prevalecerá el resultado de la defensa oral descrita en el punto 3.

# 4. Rúbrica de evaluación

El resultado de aprendizaje se puntúa sobre el 100 % repartiendo ese 100 % **a partes iguales** entre sus dos criterios de evaluación. Cada criterio se valora en cinco niveles de logro: **0 %, 25 %, 50 %, 75 % y 100 %**, de menor a mayor grado de cumplimiento.

Para asegurar que la solución entregada cumple **todos** los requisitos del enunciado, la evaluación se realiza en tres pasos: primero se comprueban los requisitos previos de entrega (apartado 4.1); después se revisa, uno por uno, cada punto de verificación de los dos criterios (apartados 4.3 y 4.4), y por último se asigna el nivel de logro de cada criterio a partir de esos puntos (apartados 4.2 y 4.5). La columna «Req.» indica el requisito del apartado 2 con el que se corresponde cada punto («Restr.» remite a las restricciones importantes y «Entrega» al apartado 3).

## 4.1. Requisitos previos de entrega (obligatorios para poder aplicar la rúbrica)

| Cód. | Requisito | Cumple |
|---|---|---|
| E1 | Se ha indicado en la tarea del Aula Virtual la URL de un repositorio **público** de GitHub, accesible, que contiene el fichero o ficheros `.py` de la solución. | □ Sí □ No |
| E2 | El repositorio contiene un `requirements.txt` que incluye `rich`; en un ordenador distinto se crea un entorno virtual, se ejecuta `pip install -r requirements.txt` y el programa arranca y funciona sin errores. | □ Sí □ No |
| E3 | El historial de commits refleja un desarrollo **incremental** desde el inicio (varios commits en fechas distintas, con mensajes descriptivos), y no un único commit con la solución terminada. | □ Sí □ No |
| E4 | Se ha creado la carpeta de pruebas pedida: archivos de varios tipos (incluido al menos uno **sin extensión**), al menos **dos niveles de subcarpetas anidadas** que ya contienen archivos y al menos **dos archivos con el mismo nombre** en carpetas distintas. | □ Sí □ No |
| E5 | Se adjunta un vídeo con, como mínimo, las cuatro ejecuciones del punto 4 de la entrega: (1) origen vacía, (2) origen que no existe o no es una carpeta, (3) destino situado dentro del origen y (4) origen y destino válidos. En cada una se ve la entrada y salida de la consola y el **estado inicial y final** de las carpetas. | □ Sí □ No |
| E6 | Se supera la defensa oral: el alumno o alumna explica dos o tres funciones elegidas por el profesorado, realiza en directo la modificación guiada y sabe justificar por qué `historial` se modifica sin `return` (lista mutable, paso «por referencia») mientras que el total de bytes debe devolverse con `return` (entero inmutable, paso «por valor»). | □ Sí □ No |
| E7 | Se cumplen las condiciones de autoría del apartado 3.1 (trabajo individual, sin similitud injustificada con otras entregas, sin código generado con IA no autorizado y con las fuentes citadas). | □ Sí □ No |

Todos los requisitos E1–E7 son obligatorios. Si alguno no se cumple, la práctica no se corrige con la rúbrica y se califica como **no superada**, de acuerdo con las condiciones del apartado 3.1.

## 4.2. Criterios correspondientes al RA3 (cada criterio: 50 %)

| Criterio | Descripción | 0 % | 25 % | 50 % | 75 % | 100 % |
|---|---|---|---|---|---|---|
| a | Se han codificado funciones para resolver problemas de matemática elemental (puntos A1–A10) | No se han definido funciones propias con `def` para los cálculos, o ninguna funciona (los valores están escritos directamente en el código), o la conversión de tamaños se delega en `rich.filesize` u | Fallan dos o más puntos esenciales: por ejemplo, `tamano_legible()` no cambia nunca de unidad, no se realizan las comprobaciones de las rutas y las funciones de cálculo | Falla un único punto esencial (por ejemplo, `una_dentro_de_otra()` considera anidadas `fotos` y `fotos2`, falta una de las tres comprobaciones de las rutas o la consola se declara como | Se cumplen todos los puntos esenciales (A1, A2, A4–A8) y, como máximo, dos de los no esenciales (A3, A9, A10) se cumplen solo parcialmente o no se cumplen (por ejemplo, pruebas insuficientes de | Se cumplen todos los puntos A1–A10: las funciones de cálculo y de comprobación son correctas, reciben sus datos por parámetro y devuelven el resultado con `return` sin imprimir nada, la presentación con `rich` está separada del cálculo y usa una única consola pasada por parámetro, el informe tiene las |

| Criterio | Descripción | 0 % | 25 % | 50 % | 75 % | 100 % |
|---|---|---|---|---|---|---|
| | | otra utilidad equivalente | imprimen en lugar de devolver el resultado. Solo alguna función del criterio funciona correctamente | variable global), o se cumplen todos los esenciales pero fallan los tres no esenciales | `tamano_legible()` o docstrings incompletos) | dos tablas pedidas y todo el código está documentado y probado, incluidos casos límite |
| b | Se ha utilizado el concepto de iteración y recursividad (puntos B1–B9) | No existe ninguna función recursiva, los archivos de las subcarpetas no se mueven, o el recorrido se resuelve con `os.walk()`, `rich.tree` u otra utilidad prohibida (incumplimiento de B7) | Fallan dos o más puntos esenciales: por ejemplo, la función recursiva no tiene caso base (provoca `RecursionError`) o en realidad no llega a llamarse a sí misma, o no se ha programado `contar_archivos_recursivo()` y además se sobrescriben archivos repetidos | Falla un único punto esencial (por ejemplo, solo funciona con un nivel de subcarpetas, no se acumulan los bytes de las llamadas recursivas, `historial` se devuelve con `return` o se crea dentro de la función, o `nombre_disponible()` no evita sobrescribir), o se cumplen todos los esenciales pero fallan los dos no esenciales | Se cumplen todos los puntos esenciales (B1–B5, B7 y B8) y uno de los no esenciales (B6 o B9) se cumple solo parcialmente o no se cumple (por ejemplo, faltan los comentarios de caso base y caso recursivo, o falla algún caso límite como una carpeta de origen vacía) | Se cumplen todos los puntos B1–B9: las dos funciones recursivas recorren cualquier nivel de subcarpetas sin `os.walk()`, con caso base y caso recursivo identificados, acumulan el valor devuelto por las llamadas recursivas y comparten la misma lista `historial` (paso «por referencia»), combinadas de forma coherente con bucles `for` (recorrido de cada carpeta e informe) y `while` (búsqueda de un nombre libre), y el programa resuelve correctamente el caso válido y los casos límite |

## 4.3. Puntos de verificación del criterio a) — Funciones de cálculo, comprobación y presentación

| Cód. | Punto de verificación | Req. | Esencial | Cumple |
|---|---|---|---|---|
| A1 | `obtener_extension(nombre_archivo)` usa `os.path.splitext()` y devuelve la extensión en minúsculas y sin el punto (`"Informe.PDF"` → `"pdf"`); si el archivo no tiene extensión devuelve `"sin_extension"`. | 3 | Sí | □ Sí □ Parcial □ No |
| A2 | `tamano_legible(tamano_bytes)` divide entre 1024 dentro de un bucle mientras el valor lo permite, contempla las cuatro unidades (bytes, KB, MB y GB) y devuelve una cadena ya formateada (p. ej. `"2.35 MB"`). No usa `rich.filesize` ni ninguna utilidad que haga la conversión por sí sola. | 4 | Sí | □ Sí □ Parcial □ No |
| A3 | `tamano_legible()` se ha probado con valores que caen en cada una de las cuatro unidades, incluidos valores límite (por ejemplo 0, 1023, 1024, 1048576 y 1073741824), y los resultados son correctos. | 4 | No | □ Sí □ Parcial □ No |
| A4 | `una_dentro_de_otra(ruta_a, ruta_b)` compara las rutas absolutas (`os.path.abspath()`) con `os.sep` añadido al final, usa `.startswith()` en ambos sentidos y devuelve `True`/`False`: `True` si son la misma carpeta o una está dentro de la otra, y `False` para casos como `fotos` y `fotos2`. | 5 | Sí | □ Sí □ Parcial □ No |
| A5 | Las rutas de origen y destino se piden con `Prompt.ask()` y texto en color, y se realizan las tres comprobaciones: el origen existe y es carpeta; el destino, si existe, es carpeta; origen y destino no son la misma ni están una dentro de la otra (usando `una_dentro_de_otra()`). Si alguna falla se muestra un aviso en **rojo** y el programa termina sin traza de error (*traceback*). | 2 | Sí | □ Sí □ Parcial □ No |
| A6 | Se crea **un único** objeto `Console` al inicio, que se pasa como parámetro a las funciones de presentación (no es una variable global) y sustituye a `print()` en toda la salida. `mostrar_cabecera(consola)` muestra el título dentro de un `Panel`. | 1, Restr. | Sí | □ Sí □ Parcial □ No |
| A7 | `mostrar_informe(consola, movidos, tamano_total, en_origen, en_destino, historial)` recibe todos los datos por parámetro y construye dos `Table`: la primera con las cuatro filas pedidas (archivos movidos, tamaño total formateado con | 10 | Sí | □ Sí □ Parcial □ No |

| Cód. | Punto de verificación | Req. | Esencial | Cumple |
|---|---|---|---|---|
| | `tamano_legible()`, archivos que quedan en el origen y archivos en el destino) y la segunda con una fila numerada por cada movimiento, recorriendo `historial` con un bucle `for`. | | | |
| A8 | Separación entre cálculo y presentación: las funciones de los puntos 3 a 9 no imprimen nada ni usan `rich`; reciben sus datos por parámetro y devuelven el resultado con `return` (o modifican sus parámetros). Solo `mostrar_cabecera()`, `mostrar_informe()` y el programa principal usan `rich`. | 11 | Sí | □ Sí □ Parcial □ No |
| A9 | El programa principal sigue el orden lógico (cabecera → petición de rutas → comprobaciones → creación de `historial` → clasificación → recuento → informe) y obtiene el número de archivos movidos con `len(historial)`. El uso de `consola.status()` es opcional y no se penaliza su ausencia. | 10 | No | □ Sí □ Parcial □ No |
| A10 | Documentación: docstring de módulo con el propósito del programa, docstring en **cada** función (qué hace, qué parámetros recibe y qué devuelve) y comentarios que explican los bloques principales. | 12 | No | □ Sí □ Parcial □ No |

## 4.4. Puntos de verificación del criterio b) — Iteración, recursividad y paso de parámetros

| Cód. | Punto de verificación | Req. | Esencial | Cumple |
|---|---|---|---|---|
| B1 | `nombre_disponible(ruta_destino, nombre_archivo)` separa nombre y extensión con `os.path.splitext()` y, mediante un bucle `while` con `os.path.exists()`, prueba `nombre_1.ext`, `nombre_2.ext`, … hasta encontrar un nombre libre. Ningún archivo se sobrescribe ni se pierde. | 6 | Sí | □ Sí □ Parcial □ No |
| B2 | `clasificar_archivo(ruta_origen, nombre_archivo, ruta_destino, historial)` obtiene la subcarpeta con `obtener_extension()`, la crea con `os.makedirs(..., exist_ok=True)`, obtiene un nombre libre con `nombre_disponible()`, mueve con `shutil.move()` y devuelve con `return` el tamaño obtenido con `os.path.getsize()` **antes** de mover. | 7 | Sí | □ Sí □ Parcial □ No |
| B3 | Paso «por referencia»: la lista `historial` se crea **antes** de la primera llamada (en el programa principal); `clasificar_archivo()` le añade con `.append()` un texto origen → destino (p. ej. `carpeta_prueba/trabajos/notas.txt → txt/notas_1.txt`) y ninguna función la devuelve con `return` ni la reasigna (`historial = ...`). | 7, 8 | Sí | □ Sí □ Parcial □ No |
| B4 | `clasificar_carpeta_recursivo(ruta_origen, ruta_destino, historial)` recorre `os.listdir()` con un bucle `for`; si el elemento es una carpeta (`os.path.isdir()`) **se llama a sí misma**, y si es un archivo llama a `clasificar_archivo()`. Pasa la **misma** lista `historial` en todas las llamadas y devuelve la suma de los bytes de sus archivos y de sus llamadas recursivas. | 8 | Sí | □ Sí □ Parcial □ No |
| B5 | `contar_archivos_recursivo(ruta)` es recursiva, recorre cualquier nivel de subcarpetas y devuelve el número total de archivos; se usa tras la clasificación para contar los archivos que quedan en el origen y los que hay en el destino (incluidos los que ya existían antes). | 9 | Sí | □ Sí □ Parcial □ No |
| B6 | En las dos funciones recursivas se identifican con comentarios el **caso base** y el **caso recursivo**. | 8 | No | □ Sí □ Parcial □ No |
| B7 | No se usa `os.walk()`, `glob` con `recursive=True`, `Path.rglob()`, `rich.tree` ni ninguna otra utilidad que resuelva el recorrido por sí sola, y las carpetas de origen no se borran aunque queden vacías. | 8, 9, Restr. | Excluyente | □ Sí □ No |
| B8 | Caso válido (similar al ejemplo orientativo): tras la ejecución quedan 0 archivos en el origen (solo carpetas vacías), todos los archivos están en el destino en subcarpetas por extensión (incluida `sin_extension`), los nombres repetidos se conservan con sufijo `_1`, `_2`… y el informe es coherente (archivos movidos = archivos iniciales y tamaño total = suma real de los tamaños). | Enunciado, 10 | Sí | □ Sí □ Parcial □ No |
| B9 | Casos límite: con la carpeta de origen vacía (o con subcarpetas vacías) el programa termina sin errores e informa de 0 archivos movidos; si el destino ya existe y contiene archivos, no se sobrescribe ninguno y se incluyen en el recuento del destino. | Entrega 4 | No | □ Sí □ Parcial □ No |

Un punto se marca como «Sí» solo cuando se cumple por completo y puede comprobarse en el código, en el vídeo, en la ejecución por el profesorado o en la defensa; «Parcial» cuando está programado pero con algún error o carencia, y «No» cuando falta o no funciona. Los elementos que el enunciado presenta como opcionales o recomendados (por ejemplo, `consola.status()` o resaltar en verde o rojo los archivos que quedan en el origen) no se penalizan si no aparecen.

### 4.5. Asignación del nivel de logro a partir de los puntos de verificación

| Nivel | Condición (se aplica por separado a cada criterio) |
|---|---|
| 100 % | Todos los puntos de verificación del criterio están marcados como «Sí». |
| 75 % | Todos los puntos **esenciales** están marcados como «Sí» y, como máximo, dos puntos no esenciales están marcados como «Parcial» o «No» (en el criterio b, como máximo uno). |
| 50 % | Un único punto esencial está marcado como «Parcial» o «No», o todos los esenciales están marcados como «Sí» pero fallan todos los no esenciales. |
| 25 % | Dos o más puntos esenciales están marcados como «Parcial» o «No», pero al menos una de las funciones del criterio está programada y funciona correctamente. |
| 0 % | Ninguna de las funciones del criterio está programada o funciona, o (solo en el criterio b) se incumple el punto excluyente B7. |

## 5. Cálculo de la nota de la práctica

La nota final de la práctica se obtiene sumando el porcentaje alcanzado en el criterio a) y en el criterio b) (50 % cada uno), según el nivel de logro descrito en la rúbrica anterior:

**Nota (%) = 0,5 × nivel del criterio a) + 0,5 × nivel del criterio b)**

Por ejemplo, con un 75 % en el criterio a) y un 100 % en el criterio b), la nota es 0,5 × 75 + 0,5 × 100 = 87,5 %, es decir, un 8,75 sobre 10.

Las respuestas de la defensa oral se tienen en cuenta en los puntos de verificación: si el alumno o alumna no es capaz de explicar una función o de modificarla en directo, los puntos asociados a esa función se marcarán como «No», con independencia de que el código entregado funcione. Si alguno de los requisitos previos del apartado 4.1 no se cumple, la práctica se califica como no superada.